

# Enforcing Fortress' Return Type Rule at Runtime

Karl Naden

Oracle Labs

karl.naden@oracle.com

## Abstract

This is a write-up of the theory behind enforcing the return type rule during runtime type instantiation in the Fortress programming language. The enforcement of this rule is required for type safety.

## 1. Introduction

The Fortress programming language has support for symmetric multiple dispatch over overloaded generic functions and methods. Unlike many of today's languages with generics, Fortress does not follow a type erasure semantics. Instead generic type information is recorded at runtime, allowing dispatch decisions to be made based on generic types. The price that we must pay for keeping this additional information is that in order to select a generic function during dispatch, we must choose an applicable and typesafe instantiation of that generic function.

Elsewhere, we have developed an algorithm and syntactic restrictions on type parameters to generic functions that supports relatively efficient determination of an applicable instantiation of a generic function if one exists. However, the algorithm does not currently enforce the return type rule which guarantees that the type of the object returned by the chosen instantiation is a subtype of the type of the function call given by static checking. The static component of the return type rule provides strong guarantees about the existence of an instantiation that is both applicable and type safe if an applicable instantiation exists. To enforce it at runtime, extra type variable bounds are needed. These can be added to and evaluated with the bounds needed to check applicability without changing which function definition will be chosen.

This write up contributes three things:

- A proof that adding bounds generated by the return type rule does not change the choice whether a dispatch semipredicate determines a function entry point to be applicable.
- An algorithm for determining the additional bounds needed utilizing the constraint generation subtyping algorithm used by the Fortress typechecker. (Not Done)
- A sketch of how to interpret the bounds such that they can be added to the open coding of the dispatch semipredicate. (Not Done)

|                    |        |       |                                                                           |
|--------------------|--------|-------|---------------------------------------------------------------------------|
| Concrete Type      | $T$    | $::=$ | $t[\overline{T}] \mid \text{Object}$                                      |
| Generic Type       | $\tau$ | $::=$ | $X \mid t[\overline{\tau}] \mid T$                                        |
| Type Declaration   | $d$    | $::=$ | $\text{trait } t[\overline{V} \ tp] \text{ extends } t'[\overline{\tau}]$ |
| Variance           | $V$    | $::=$ | $\text{covariant} \mid \text{invariant}$                                  |
| Type Parameter     | $tp$   | $::=$ | $X \text{ extends } \tau$                                                 |
| Function Signature | $fn$   | $::=$ | $f[\overline{tp}](\overline{y} : \overline{\tau}) : \tau$                 |

Figure 1. Syntax.  $t$ ,  $f$ ,  $X$ , and  $y$  are identifiers

## 2. Background

This work builds on what is already a very complex formulation of the Fortress programming language. A formal presentation of this system can be found elsewhere. Here we provide informal and simplified descriptions of the features of the language needed to present our problem and solution. In particular, we consider the subset of the language with nominal generic traits, generic overloaded functions, and symmetric multiple dispatch.

### 2.1 Fortress Types

Fortress is an object oriented programming language with a nominal type system. Since our purpose is to study the interaction between the type hierarchy and function dispatch, we dispense with the details of objects and simply abstract them to *traits* defining a nominal type and its place in the subtyping lattice.

#### 2.1.1 Declared Types

A concrete type  $T$  is either the distinguished top type `Object`, which is a supertype of all types, or a trait  $t$  with concrete types  $\overline{T}$  provided for each of its static parameters, written  $t[\overline{T}]$ . We call this an *instantiation* of  $t$ . In contrast, a generic type  $\tau$  is a raw type variable  $X$ , a trait instantiated at generic types  $t[\overline{\tau}]$ , or a concrete type  $T$ . Each trait is defined by a name  $t$ , a possible empty list of static type parameters  $\overline{X}$  enclosed in oxford brackets, and an optionally empty list of supertraits  $t'[\overline{\tau}]$ . Each type parameter includes a *variance* and a list of upper bounds. The variance  $V$  controls subtyping between different instantiations of the same trait as described below. An upper bound  $\tau$  on type variable  $X$  is a generic type which can only include type parameters declared before  $X$ . In other words, scoping of type parameters is from left to right. An instantiation  $t[\overline{T}]$  where  $T_X$  is given for  $X$  meets the bound  $\tau$  on  $X$  if  $T_X$  is a subtype of  $\tau$  when the instantiating types are substituted for their corresponding type variables in  $\tau$ . Formally,  $T_X <: \tau[\overline{T}/\overline{X}]$ . The list of supertraits of a trait may also be generic in the type parameters declared for the trait.

#### 2.1.2 Subtyping

The subtype relationship  $<:$  is defined as the reflexive and transitive closure of three rules:

1. (Object): All types are subtypes of `Object`,

2. (Variance): For two valid instantiations of the same trait, we have  $t[\overline{T}] <: t[\overline{S}]$  if and only if for each type parameter  $X$  of  $t$  with defined variance  $V_X$ ,  $T_X \equiv S_X$  if  $V_X$  is *invariant* and  $T_X <: S_X$  if  $V_X$  is *covariant*.
3. (Trait): If  $t'[\overline{\tau}]$  appears in the `extends` clause of trait  $t[\overline{X}]$ , then  $t[\overline{T}] <: t'[\overline{\tau}][\overline{T}/\overline{X}]$ .

### 2.1.3 Structural Types

In order to ensure that the universe of types forms a meet-bounded lattice (important for symmetric multiple dispatch described below), Fortress includes union and intersection types which are collections of concrete types representing the join and meet on the declared type lattice. These types are not represented in the Fortress syntax, but can be inferred or appear at runtime. Subtyping is defined as normal for union and intersection types.

## 2.2 Functions

This paper only considers function dispatch and so we ignore function bodies and instead describe only the signatures of functions and the restriction placed on functions with the same name.

### 2.2.1 Function Signatures

In our system each function signature represents an *entry point* or concrete piece of code that can be executed when the named function is called and specifies all of the type information needed for typechecking. A function signature  $f[\overline{tp}](\overline{y} : \overline{\tau}) : \tau_r$  contains a list of static parameters  $\overline{tp}$ , the names  $\overline{y}$  and corresponding types  $\overline{\tau}$  of the inputs to the function, and a return type  $\tau_r$ . The static parameters of functions include bounds and are scoped just as they are for traits. We can provide a list of concrete types  $\overline{T}$  to create an instantiation  $f[\overline{T}]$  of a function. Valid instantiations of functions must meet the bounds on the type parameters as for valid instantiations of traits.

Given a list of types  $(\overline{S})$  and a concrete instantiation  $f[\overline{T}]$  of a function  $f[\overline{tp}](\overline{y} : \overline{\tau}) : \tau_r$  with type parameters  $\overline{X}$  we say that  $f[\overline{T}]$  is *applicable* to inputs of these types if for each input  $y_i$  we have  $S_i <: \tau_i[\overline{T}/\overline{X}]$ . We also say that a generic function signature  $f$  is applicable to  $(\overline{S})$  when such an instantiation of  $f$  exists.

We also define a specificity relationship on pairs of function signatures. Only function signatures with the same name and number of arguments can be related. In this case, if signature  $fn_1$  is applicable to a subset of the argument types to which signature  $fn_2$  is applicable, then  $fn_1$  is *more specific* than  $fn_2$ , written  $fn_1 \preceq fn_2$ . We are not concerned with the formal definition which is discussed in depth elsewhere.

### 2.2.2 Function Overloading

A key feature of the Fortress programming language is symmetric multiple dispatch on overloaded functions. In short this allows programmers to define multiple entry points for a single function name and when a function is called the runtime will choose the most specific entry point that is applicable to the provided arguments. A set of function signatures with the same name and number of parameters is called an *overload set*. To ensure that there is a unique most specific entry point for each set of arguments, several restrictions must be placed on overload sets.

1. (No Duplicates): If  $fn_1 \preceq fn_2$  and  $fn_2 \preceq fn_1$  then  $fn_1 = fn_2$ .
2. (Meet): For any  $fn_1$  and  $fn_2$  from a single overload set there exists a signature from the overload  $fn_0$  such that  $fn_0 \preceq fn_1$ ,  $fn_0 \preceq fn_2$ , and  $fn_0$  is applicable to all argument types to which both  $fn_1$  and  $fn_2$  are applicable.

A third restriction is also needed to ensure type safety. For each function call the Fortress type checker identifies the most specific

entry point that must be applicable based on static information. The runtime system has access to the actual types of the arguments provided to this call and so may be able to choose a more specific entry point. For type safety, the return type of the more specific entry point must be a subtype of the statically guaranteed return type. This gives rise to the return type rule:

3. (Return): If  $fn_1 \preceq fn_2$  and  $fn_1$  is applicable to  $(\overline{T})$  then for every instantiation of  $fn_2$  that is applicable to  $(\overline{T})$  with return type  $T_2$ , there exists an instantiation of  $fn_1$  that is applicable to  $(\overline{T})$  and has return type  $T_1$  where  $T_1 <: T_2$ .

Taken together, these rules have been proven to be sufficient to ensure that Fortress' symmetric multiple dispatch is unambiguous and type safe.

### 2.2.3 Dispatch Semi-predicate

In order to implement symmetric multiple dispatch, the runtime system must analyze the runtime types of the arguments provided to a function call and find the most specific applicable entry point. Furthermore because generic types must be reified, if the chosen entry point is a generic function definition the runtime must choose an appropriate instantiation. The instantiation must be applicable to the runtime argument types and also follow the return type rule.

In previous work, we developed a *dispatch semipredicate* that formalizes the runtime choice. Given a function signature  $fn$  and a set of argument types  $(\overline{T})$  the semipredicate returns a list of types that witness an instantiation of  $fn$  that is applicable to  $(\overline{T})$ , or returns false if there is no such instantiation.

We also developed an algorithm to compute the semipredicate. The algorithm is based on the fact that one way to find a witness or prove that one does not exist is to generate subtyping constraints on the instantiation of the type variables of  $fn$ . If these constraints are mutually satisfiable, then the semipredicate succeeds and otherwise fails. Constraints can come from two places.

1. (Applicability) constraints needed to ensure the function parameter types are supertypes of the provided argument types.
2. (Relationship) constraints needed to satisfy the declared relationships between type parameters.

Our original algorithm takes advantage of several properties of the system. First, it is easy to ask a ground type what its super types are at runtime which means that constraints of the form  $T <: \tau$  and  $T \equiv \tau$  can be checked and simplified easily. Second, because our system only supports invariant and covariant generics, applicability constraints will only ever be of this form. Third, because we are looking for the most specific instantiation we want to choose the most specific type for each variable. This means that once we have generated all of the lower bounds for a type variable, we can use it as the presumable instantiation. The scoping of the type parameters from left to right ensures that the right most type parameter will have no lower bounds generated by relationship constraints. Fourth, type parameters have upper bounds only.

Our algorithm first generates applicability constraints. Next, we calculate the lower bound on the rightmost type parameter  $X_n$  from the meet of its applicability bounds and use it to generate relationship bounds with any generic upper bounds. Since  $X_n$  is not in scope in its own bound, no new bounds are generated for it meaning that its instantiation is still valid. We continue processing the type variables in reverse order and at the end we have the most specific applicable instantiation. The algorithm is nice because it requires only a single pass for relationship constraints. Note also that if at any point a subtyping constraint is known to be unsatisfiable, the algorithm returns false.

Unfortunately, this algorithm does not take into account the return type rule. The return type rule essentially adds another source of restrictions on type variables.

3. (*Return Type*) constraints needed to ensure that the return type of the instantiated function is a subtype of the statically guaranteed return type.

However, in this case, the constraint to simplify is of the form  $\tau <: T$ . Secondly, it is not obvious where these constraints should be added to the algorithm. The contribution of this paper is to explain how to simplify these constraints to usable bounds and where they can be added into the algorithm without impacting its basic form.

### 3. Placing Return Type Restrictions

To add support for the return type rule to the dispatch semipredicate, restrictions generated by the return type must be incorporated into the algorithm. It would be nice if we could add the constraints in with the applicability constraints because then they would be folded in automatically and not impact the single pass for the relationship constraints. However, the return type rule indicates that the return type restrictions does not impact applicability. Therefore, we might worry that evaluating return type constraints prior to being sure of applicability might prevent us from finding an applicable and type safe instantiation were we to wait.

Fortunately, we can prove that the guarantees provided by the return type rule are strong enough to prevent this situation. The following theorem shows this:

**Theorem 3.1.** *Let  $f_{n_1} \preceq f_{n_2}$  be members of the same overload set. Take a call site at which  $f_{n_2}$  is statically known to be applicable to static argument types  $(\overline{T})$  with return type  $T_2$ , but  $f_{n_1}$  is not applicable. If there exists no instantiation of  $f_{n_1}$  such that its return type  $T_1$  satisfies  $T_1 <: T_2$ , then  $f_{n_1}$  cannot be applicable to any list of types  $(\overline{S}) <: (\overline{T})$ .*

*Proof.* Since  $(\overline{S}) <: (\overline{T})$ , we know that the given static instantiation of  $f_{n_2}$  is applicable to  $(\overline{S})$ . By the return type rule, if an instantiation of  $f_{n_1}$  that is applicable to  $(\overline{S})$  exists, then there must exist some instantiation of  $f_{n_1}$  that is applicable and has a more specific return type than  $T_2$ . However, we assumed that this is not the case which means that  $f_{n_1}$  cannot be applicable.  $\square$

Practically this means that return type restrictions are closely tied to applicability constraints and one set is satisfiable if and only if the other is. Therefore, if we find that a return type restriction cannot be satisfied then we can safely return false immediately as with applicability constraints. Thus, return type constraints can be generated and added before relationship constraints are evaluated maintaining the single pass algorithm.